Informe Parcial 2

Adrien Roho, Bryan Llontop, Diego Castillo
Universidad Nacional de Colombia, Sede Bogotá
Bogotá, Colombia
aroho@unal.edu.co
bllontop@unal.edu.co
dcastillo@unal.edu.co

Abstract — Este informe describe una aplicación en Java 21 que gestiona tareas de análisis de partículas (contexto I.2) mediante cinco componentes implementados desde cero: un árbol general, un árbol binario de búsqueda (BST), un árbol AVL, un max-heap binario y Heap-Sort. Se presentan el diseño de las estructuras, una tabla de complejidades, la comparación de alturas entre BST y AVL (6 frente a 2 con las claves 1..7, y 999 frente a 9 con 1..1000) y el mecanismo de cancelación en cascada que mantiene la coherencia entre estructuras. **Este informe fue redactado por una inteligencia artificial y corregido manualmente por los autores del documento.** Los resultados citados provienen de la ejecución real de los casos A–F sobre el código entregado.

Palabras clave — árbol general, BST, AVL, max-heap, Heap-Sort, cola de prioridad, cancelación en cascada.

I. INTRODUCCIÓN

El examen propone construir un programa pequeño que demuestre el uso correcto de estructuras de datos jerárquicas y de prioridad. Elegimos el contexto **I.2 (Simulación de partículas)**: cada registro es una tarea de análisis de una partícula, y su prioridad (1 a 10, donde 10 es más urgente) indica la urgencia del análisis. No se simula física alguna; los datos son inventados.

Cada **registro** tiene id (entero positivo único), nombre, prioridad, padre (0 indica raíz) y estado (PENDIENTE, ATENDIDO o CANCELADO). Las reglas comunes son: no hay IDs repetidos, el padre debe existir, no hay ciclos, hay una sola raíz, y los registros permanecen en el BST y el AVL como historial. El desempate de prioridades es: mayor prioridad primero y, si empatan, menor ID.

El programa ofrece seis operaciones por consola: REGISTRAR, CONSULTAR, SIGUIENTE, CANCELAR, REPORTE y ESTADISTICAS. Se añadió además ELIMINAR id, que realiza el borrado explícito en BST y AVL exigido por el caso E.

El objetivo de este informe es (1) presentar el problema y la arquitectura, (2) comparar complejidades, (3) contrastar BST y AVL y (4) explicar cómo se cancela una tarea sin perder coherencia, junto con los errores encontrados y las decisiones tomadas.

II. DESARROLLO DE CONTENIDOS

A. Problema elegido y diagrama de estructuras

Todas las estructuras comparten **los mismos objetos Registro**. Por eso, cuando cambia el estado de un registro, el cambio es visible de inmediato en todas ellas. Cada estructura cumple un papel distinto:

Estructura	**Clase**	**Rol en el programa**
Árbol general	ArbolGeneral	Jerarquía padre–hijo; recorridos, altura, hojas y cancelarSubarbol. Es la fuente de verdad de los pendientes.
BST	BST	Índice por id sin balanceo; conserva el historial y sirve de contraste con el AVL.
AVL	AVL	Índice por id balanceado;

Estructura	Clase	Rol en el programa
		atiende CONSULTAR.
Max-heap	BinaryHeap	Cola de atención (SIGUIENTE).
Heap-Sort	HeapSort	Ordena una **copia** de los pendientes para REPORTE.

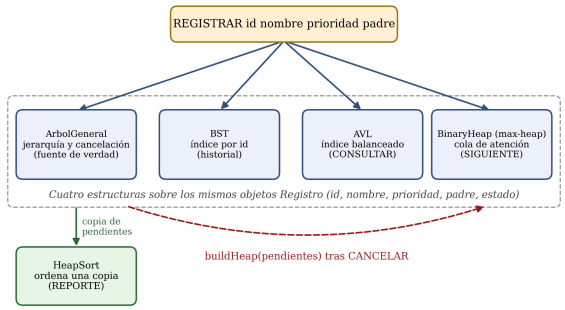


Fig. 1. Los cuatro índices comparten los mismos objetos Registro. El árbol general reconstruye el heap tras cada cancelación.

B. Tabla de complejidades

TABLA I. COMPLEJIDAD TEMPORAL (n = número de registros)

Operación	Caso esperado	Peor caso
Buscar en BST	O(log n)	**O(n)** (árbol degenerado)
Buscar en AVL	O(log n)	O(log n)
Insertar en AVL	O(log n)	O(log n) (a lo sumo una rotación simple o doble)
Insertar en heap	O(1) amortizado / O(log n)	O(log n)
Extraer máximo del heap	O(log n)	O(log n)
Construir heap (buildHeap)	O(n)	O(n)
Heap-Sort	O(n log n)	O(n log n)

buildHeap es O(n) porque la mayoría de los nodos están cerca de las hojas y su siftDown recorre pocos niveles; la suma de los costos converge a n. Heap-Sort construye el heap en O(n) y realiza n extracciones de O(log n), por lo que es O(n log n) incluso en el peor caso. A diferencia del BST, el AVL conserva O(log n) siempre porque su altura está acotada por aproximadamente $1{,}44 \cdot \log_2 n$.

C. Comparación BST–AVL

Por qué las claves ordenadas perjudican al BST y no al AVL. En un BST la forma depende del orden de inserción. Si las claves llegan ordenadas (1, 2, 3, …), cada nueva clave es mayor que todas las anteriores y se inserta siempre como hijo derecho del último nodo. El árbol degenera en una lista enlazada de altura n−1, y buscar, insertar o eliminar cuestan O(n) en lugar de O(log n). El AVL inserta igual, pero después de cada inserción recalcula alturas y el factor de balance de los ancestros. Si algún nodo queda con factor distinto de −1, 0 o 1, aplica una rotación (LL, RR, LR o RL) que restaura el equilibrio sin alterar el orden inorden. Así la altura permanece en O(log n) sin importar el orden de llegada, a costa de un pequeño trabajo extra por inserción.

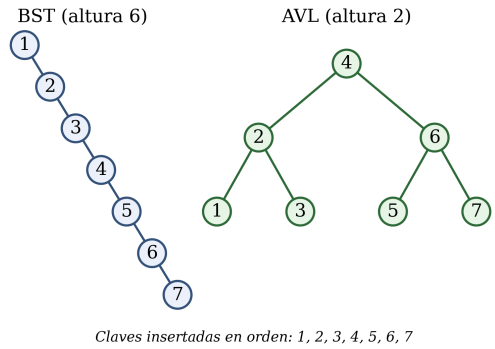


Fig. 2. Árboles resultantes de insertar 1,2,…,7 en orden: el BST degenera en una cadena (altura 6) y el AVL queda completo (altura 2).

Alturas medidas. Convención: árbol vacío = −1, hoja = 0.

TABLA II. ALTURAS INSERTANDO CLAVES ORDENADAS 1..n (SALIDA REAL DEL PROGRAMA)

n	Altura BST	Altura AVL	Raíz del AVL
7	6	2	4
1000	999	9	—

Con n = 7 el AVL queda como un árbol completo de altura 2 con raíz 4. Con n = 1000 la diferencia es de 999 frente a 9 niveles ($\lfloor \log_2 1000 \rfloor = 9$), lo que ilustra la ventaja del AVL en el peor caso. En ambos casos el inorden es 1,2,…,n y el AVL queda balanceado. Tras eliminar la clave 4 (caso E), la clave deja de aparecer en la búsqueda y en el inorden de ambos árboles; el BST baja a altura 5, y el AVL se mantiene en altura 2 con raíz 5 y balanceado.

Rotaciones (caso D). Con tres claves, las cuatro inserciones producen la misma raíz final (20), inorden 10,20,30 y altura 1:

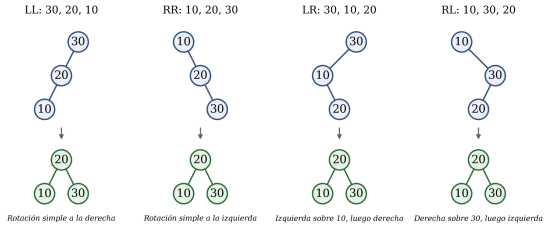


Fig. 3. Las cuatro rotaciones del caso D: árbol antes (azul) y después (verde) de balancear.

TABLA III. ROTACIONES AVL DEL CASO D

Caso	Inserciones	Desbalance en	Rotación aplicada
LL	30, 20, 10	30 (hijo izquierdo, subárbol izquierdo)	Rotación simple a la derecha sobre 30
RR	10, 20, 30	10 (derecha, derecha)	Rotación simple a la izquierda sobre 10
LR	30, 10, 20	30 (izquierda, derecha)	Izquierda sobre 10, luego derecha sobre 30
RL	10, 30, 20	10 (derecha, izquierda)	Derecha sobre 30, luego izquierda sobre 10

D. Cancelación, errores encontrados y decisiones tomadas

Cómo se evita atender una tarea cancelada. CANCELAR id ejecuta dos pasos:

1. cancelarSubarbol(id) recorre el subárbol y cambia a CANCELADO el estado de cada nodo que aún está PENDIENTE. Los registros ya ATENDIDO conservan su estado. Devuelve cuántos canceló.
2. reconstruirHeap recoge del árbol general solo los registros PENDIENTE y llama a buildHeap con ellos (O(n)). El heap queda sin ninguna tarea cancelada, por lo que SIGUIENTE nunca puede devolverla. Como defensa adicional, SIGUIENTE descarta cualquier candidato cuyo estado no sea PENDIENTE.

Coherencia entre estructuras. Como todas comparten el mismo objeto Registro, el estado es único: no hay copias que puedan divergir. El árbol general es la fuente de verdad de los pendientes; el heap es solo una vista derivada y se reconstruye de ella. El BST y el AVL **no** eliminan los nodos cancelados ni atendidos (regla f), de modo que CONSULTAR sigue funcionando y muestra el estado real. Además, REGISTRAR valida ID duplicado, padre inexistente, raíz única y prioridad en rango antes de modificar nada, por lo que un error no deja las estructuras a medias.

Resultado del caso A. Se registran 10 (prioridad 2), 20 (5) y 30 (3). CONSULTAR 20 muestra PRIORIDAD=5 ... ESTADO PENDIENTE; SIGUIENTE atiende a 20; CANCELAR 10 cancela 2 registros (10 y 30) y el 20 sigue ATENDIDO; REPORTE queda vacío y hay 0 pendientes. El árbol general conserva los tres nodos: preorden 10,20,30; postorden 20,30,10; por niveles 10,20,30; altura 1 y dos hojas.

Otros resultados verificados.

- *Caso B (empates):* con las prioridades 50→7, 30→7 y 40→9, el reporte es 40(9) 30(7) 50(7) y los dos SIGUIENTE atienden 40 y luego 30.
- *Caso C (errores):* se acepta el primer registro, se rechaza el segundo por ID duplicado y el tercero por padre 99 inexistente; queda un solo registro.
- *Caso F (heap):* con las prioridades [4,10,3,5,1,8,9,2], el reporte es 10, 9, 8, 5, 4, 3, 2, 1; el primer SIGUIENTE atiende la prioridad 10 y el segundo la 9. El reporte sale en orden de atención (descendente); el orden ascendente [1,2,3,4,5,8,9,10] corresponde a la lectura inversa de ese mismo resultado.

Error encontrado y corregido. Durante la implementación de CANCELAR, la IA sugirió eliminar físicamente los nodos cancelados del árbol general, del BST y del AVL. Al contrastar con la regla (f) y el ejemplo resuelto del enunciado, concluimos que era incorrecto: eliminarlos haría fallar CONSULTAR sobre un registro cancelado. Corregimos la lógica para que la cancelación solo cambie el estado y reconstruya el heap; lo comprobamos ejecutando el caso A y consultando un registro cancelado.

Decisiones de diseño.

- *Reconstruir el heap en lugar de borrar un elemento arbitrario.* El enunciado lo permite y evita un heap indexado. El costo es O(n) por cancelación, aceptable para el tamaño del problema.
- *Árbol general como fuente de verdad.* Evita mantener listas de pendientes paralelas que puedan desincronizarse.
- *Desempate en un único comparador.* Mayor prioridad primero y, a igual prioridad, menor ID; se reutiliza en el heap y en Heap-Sort para que SIGUIENTE y REPORTE sean siempre consistentes.
- *Heap-Sort sobre una copia.* REPORTE no altera el heap de atención.

III. CONCLUSIÓN

Se implementaron y probaron los cinco componentes pedidos, integrados en una aplicación de consola con las seis operaciones del enunciado. Los casos A–F dieron los resultados esperados: las alturas del BST y del AVL fueron 6 y 2 con las claves 1..7, y 999 y 9 con 1..1000; las cuatro rotaciones produjeron la raíz 20; y el heap y el reporte respetaron el orden de prioridad con desempate por menor ID.

La comparación confirma que el AVL garantiza O(log n) en el peor caso, mientras que el BST puede degenerar hasta O(n) con entradas ordenadas. En cuanto a la cancelación, la decisión clave fue conservar el historial en los índices y reconstruir únicamente el heap a partir de los pendientes del árbol general, lo que impide que una tarea cancelada sea atendida y mantiene la coherencia entre estructuras.

Como trabajo futuro se podría implementar un heap indexado para cancelar en O(log n) por elemento y permitir el cambio de prioridad, algo que el examen no exige.

REFERENCIAS

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4.ª ed. Cambridge, MA, EE. UU.: MIT Press, 2022.

[2] M. A. Weiss, *Data Structures and Algorithm Analysis in Java*, 3.ª ed. Boston, MA, EE. UU.: Pearson, 2012.

[3] S. Yepes García, "Examen 02 — ABP-IA: Árboles, BST, AVL, Heaps y Heap-Sort", Universidad Nacional de Colombia, Estructuras de Datos 2026-2, 2026.